



Department of Computer Science

Assignment 1

Taskflow: A Lightweight Parallel and Heterogeneous Task Graph Computing System

Student Name	Roll No
Raqib Hayat	NUM-BSCS-2022-40
Abu Bakar	NUM-BSCS-2022-41

Course: CS-440 Parallel and Distributed Computing

Instructor: Dr. Muzamil Ahmed

TASK 1: Problem Statement, Motivation, and Challenges

Problem Statement

The paper addresses a critical limitation in existing Task Graph Computing Systems (TGCS) for parallel and heterogeneous computing. Unlike traditional systems like oneTBB, StarPU, and HPX which have enabled parallel execution on CPUs and GPUs, three fundamental problems are faced that prevent them from fully exploiting modern hardware capabilities.

The very first reason is that existing systems rely too much on Directed Acyclic Graph (DAG) models, which cannot express the control flow decisions like conditions or loops directly within the graph. The developers must synchronize the entire graph execution with the runtime system outside the graph structure whenever they encounter conditional logic or iterative algorithms. This creates expensive synchronization overhead that could be eliminated if control flow were integrated into the graph itself.

Second, current TGCS frameworks do not align well with emerging GPU task graph parallelism, particularly CUDA Graph technology introduced in CUDA 10. CUDA Graph allows dependent GPU operations to be executed with dramatically reduced kernel launch overhead by creating an executable graph image. However, the operations of existing systems are merely as encapsulated functions within CPU tasks.

Third, most of the systems are either optimized for CPU-focused workloads or the GPU-focused workloads, but simultaneously for both very rarely. This is the limitation that becomes critical in heterogeneous computing environments where applications need to dynamically balance computation between CPUs and GPUs based on available parallelism.

Motivation and Real-World Importance

This problem is particularly important in modern scientific computing applications such as VLSI computer-aided design (CAD), machine learning, and data analytics. The authors' work is motivated by their DARPA project aimed at reducing circuit design times. Many CAD algorithms exhibit irregular computational patterns and complex control flow that require strategic task graph decompositions. For example, timing analysis algorithms may need to run thousands of incremental iterations, each involving conditional decisions about whether to use CPU or GPU for computation based on the problem size.

In machine learning workloads like sparse neural network inference, applications must iteratively process data through thousands of dependent operations across multiple GPUs. Traditional approaches that unroll loops statically or handle conditions outside the graph result in significant memory overhead and inefficient resource utilization.

Challenges in Existing Techniques

The paper identifies three key gaps in current parallel computing frameworks. First is the lack of end-to-end tasking capability. When implementing iterative algorithms using DAG models, developers must repetitively wait for task graph completion at each iteration boundary, involving expensive synchronization between application code and the runtime system.

Second, existing work-stealing schedulers are designed primarily for homogeneous CPU environments and do not effectively manage heterogeneous worker pools comprising both CPU and GPU threads. They

cannot adapt worker counts dynamically to match available task parallelism across different computational domains.

Third, current systems struggle with energy efficiency and throughput when multiple programs run concurrently. Without intelligent worker management, systems either underutilize resources when task parallelism is low or waste energy through excessive failed steal attempts when workers cannot find tasks to execute.

TASK 2: Approach, Methodology, and Techniques

Computational Model and Architecture

Taskflow introduces a Heterogeneous Task Dependency Graph (HTDG) programming model that extends beyond traditional DAG-based approaches. The system employs a task parallelism model where computation is decomposed into tasks representing function calls, and dependencies between tasks define execution order. Unlike pure data parallelism or loop parallelism, task parallelism allows Taskflow to manage irregular computational patterns common in scientific applications.

The core innovation lies in Taskflow's hybrid approach combining static task graphs with dynamic flow control. The system supports five fundamental task types that work together to enable expressive parallel programming. Static tasks represent basic computational units, dynamic tasks enable hierarchical graph construction through subflows, module tasks support compositional programming for code reusability, condition tasks introduce in-graph control flow, and cudaFlow tasks encapsulate GPU task graphs.

Programming Model and Framework Design

Taskflow leverages modern C++17 features, particularly closures and lambda functions, to create an expressive programming interface. Developers construct task graphs declaratively by emplacing tasks into a taskflow object and establishing dependencies using preceded and succeed methods. This design philosophy differs from imperative approaches used in OpenMP or explicit message passing in MPI.

Traditional parallel frameworks like MPI for distributed computing are not used in the system instead, it focuses more on shared-memory parallelism within a single heterogeneous node containing multiple CPU cores and GPUs. For CPU task execution, Taskflow implements a custom work-stealing scheduler rather than relying on external threading libraries. For GPU operations, it integrates directly with CUDA Graph API and provides two abstractions: cudaFlow for explicit graph construction when all kernel parameters are known in advance, and cudaFlowCapturer for wrapping existing stream-based GPU libraries like cuBLAS and cuDNN.

Key Technical Innovations

The conditional tasking mechanism represents a significant departure from existing TGCS approaches. A condition task returns an integer index that determines which successor task executes next, with the index corresponding to the order of successors. This simple yet powerful mechanism enables developers to express if-else decisions, iterative loops, and even non-deterministic control flow directly within the task graph structure. The system uses two types of dependencies: strong dependencies that must be satisfied for execution, and weak dependencies emanating from condition tasks that allow cyclic execution patterns.

For GPU task scheduling, Taskflow employs a greedy round-robin algorithm to transform cudaFlowCapturer graphs into executable CUDA graphs. The task graph topologically and assigning each task to one of several CUDA streams based on a configurable maximum stream count are leveled by the algorithm. Event recording and waiting mechanisms are used to manage the Cross-stream dependencies, optimizing kernel concurrency without violating task dependencies.

Separate worker pools are maintained by the heterogeneous work-stealing scheduler for each computational domain (CPU and GPU), with each worker having dedicated task queues for both domains. The scheduler implements a per-domain invariant: when active workers exist in a domain, at least one worker must be attempting steals unless all workers are active. This adaptive strategy minimizes wasteful steal attempts while ensuring resources are not underutilized when task parallelism is available.

TASK 3: System Architecture Diagram

Self-Drawn Architecture Diagram

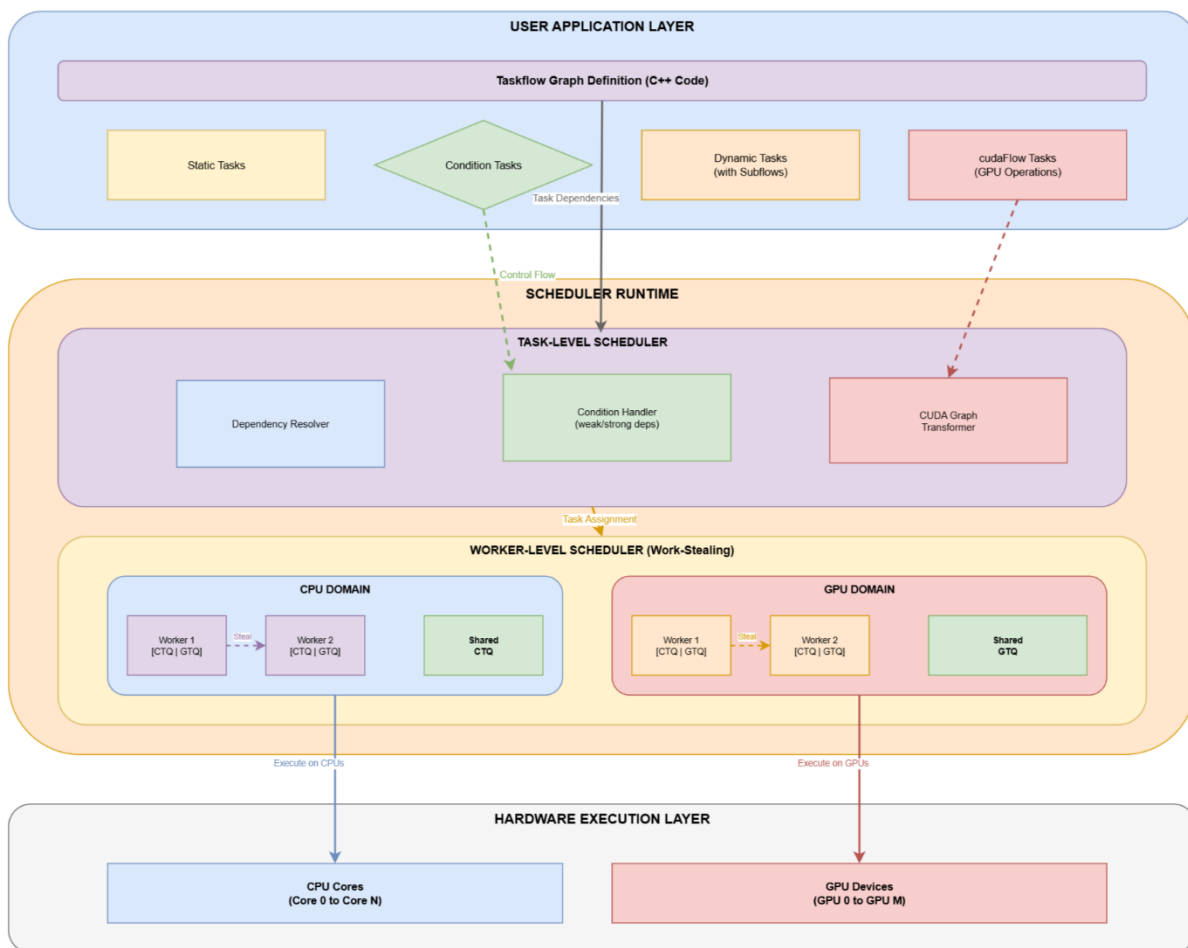


Figure 1 Diagram.

Component Explanation

User Application Layer: Developers define task graphs using Taskflow's C++ API. Different task types (static, dynamic, condition, cudaFlow) are created through `emplace` methods and connected via `preceded/succeed` to establish dependencies. This layer represents the declarative specification of parallel computation.

Task-Level Scheduler: This component manages task execution order and transformation. The Dependency Resolver tracks strong and weak dependencies, ensuring tasks execute only when dependencies are satisfied. The Condition Handler processes integer returns from condition tasks to determine next execution paths. The CUDA Graph Transformer converts `cudaFlow` and `cudaFlowCapturer` tasks into executable CUDA graphs using the revalidation algorithm.

Worker-Level Scheduler: Employs heterogeneous work-stealing among different domains. Each worker possesses two queues: CTQ (CPU Task Queue) and GTQ (GPU Task Queue), facilitating swift addition of domain-specific tasks without competition. Shared queues manage external task submissions. Workers follow the adaptive steal strategy, incrementing domain-specific atomic counters (actives, thieves) to maintain the per-domain invariant.

Data Flow: Tasks flow from user definition to the scheduler, which assigns them to appropriate domain queues. Workers steal from random victims within their domain, respecting the two-phase committed protocol to prevent underutilization. When condition tasks execute, control flows back to the dependency resolver, potentially creating cycles or branching paths. GPU tasks are batched into CUDA graphs before hardware execution, minimizing kernel launch overhead.

TASK 4: Results and Performance Metrics

Performance Evaluation Overview

Taskflow against the four baseline systems was evaluated by researchers: StarPU (system for scientific computing), oneTBB (comprehensive framework for intel), OpenMP (directive-based model), and HPX (C++ library focused on parallelism). All experiments were conducted on a heterogeneous machine that featured 4 GeForce RTX 2080 GPUs and 40 Intel Xeon CPU cores, operating on Ubuntu Linux with CUDA 11.

Micro-benchmark Results

For randomly generated DAGs with equal distribution of CPU and GPU tasks performing SAXPY operations, Taskflow demonstrated consistent advantages across multiple metrics. In terms of execution speed, Taskflow finished the largest task graph (100,000 tasks) 1.37 times quicker than oneTBB, 1.44 times faster than StarPU, 1.53 times quicker than HPX, and 1.40 times quicker than OpenMP. The analysis of runtime distribution indicated that Taskflow exhibited the lowest variance and the most reliable execution times over 100 runs.

Memory footprint measurements revealed Taskflow's efficiency in resource utilization. The system-maintained memory usage close to oneTBB and OpenMP (around 500-600 MB for large graphs), while

consuming $1.5\times$ less memory than StarPU and HPX. This advantage stems from Taskflow's conditional tasking model, which reuses condition tasks in loops rather than unrolling graphs statically.

Energy efficiency metrics measured using Linux perf tools showed Taskflow consistently consumed lower total joules across all problem sizes. For the largest graph, total energy consumption was approximately 600 joules for Taskflow compared to 800-1000 joules for baseline systems. Average power consumption remained stable around 100-120 watts, whereas StarPU and HPX showed increasing power with larger graphs.

Throughput measurements for concurrent execution (running multiple programs) demonstrated Taskflow's superior worker management. Using weighted speedup metrics, Taskflow achieved throughput values between 1.5-2.0 across 2-8 concurrent processes, outperforming oneTBB (1.2-1.5) and StarPU (0.8-1.0). This result highlights how Taskflow's adaptive worker management prevents resource waste through controlled steal attempts.

Real-World Application Performance

For **VLSI incremental timing analysis** (1000 iterations on 1.6M gate design), Taskflow completed in 39.11 minutes using 40 CPUs and 1 GPU, compared to 48.51 minutes for oneTBB and 55.43 minutes for StarPU - representing 19-29% speedup. Memory usage remained stable around 3-5 GB for Taskflow versus 10-25 GB for baseline systems that unroll iterative graphs.

For **sparse neural network inference** (1920-layer network with 4096 neurons per layer), Taskflow achieved runtime of 750 ms with 4 GPUs compared to 1451 ms for oneTBB and over 1600 ms for StarPU. The cudaFlow implementation using CUDA Graph was $1.9\times$ faster than stream-based execution with 4 streams, demonstrating the benefit of GPU task graph acceleration.

Scalability analysis showed Taskflow maintained near-linear speedup up to 16 CPUs before saturating due to application-specific irregularity, with better resource utilization efficiency than baselines at all concurrency levels.

TASK 5: Alternative Approach and Improvements

My Proposed Approach

If I were addressing the same problem of heterogeneous task graph computing with control flow, I would propose a **machine learning-based adaptive scheduling enhancement** integrated with Taskflow's existing architecture. This approach would augment rather than replace Taskflow's work-stealing scheduler with predictive intelligence for domain assignment and worker management decisions.

Core Improvement Concept

The fundamental idea is to introduce a lightweight **reinforcement learning agent** that learns optimal scheduling policies during runtime. Instead of relying solely on work-stealing heuristics, the system would predict: (1) whether a dynamically spawned task should execute on CPU or GPU based on input size and computational characteristics, (2) optimal number of active workers needed for upcoming graph sections, and (3) which victim to target during steal attempts rather than random selection.

The ML model would be a compact neural network trained online using the reward signal of task completion time, energy consumption, and successful steal rate. The state space would include features like current active worker count, queue lengths across domains, recent task execution times, and upcoming task types visible through lookahead in the task graph. Actions would represent discrete choices: CPU vs GPU assignment, number of workers to wake up, and victim selection priority.

Integration with Existing System

This approach preserves Taskflow's core strengths while adding intelligence. The conditional tasking model and HTDG structure remain unchanged - developers still write the same code. The ML agent would operate as an additional layer in the worker-level scheduler, providing hints to the work-stealing algorithm rather than replacing it entirely. If the ML model's confidence is low or predictions prove inaccurate, the system falls back to Taskflow's proven work-stealing strategy.

For GPU scheduling, the ML agent could learn patterns in cudaFlow graph structures to predict optimal stream counts for the cudaFlowCapturer transformation algorithm. Instead of using a fixed MAX_STEALS parameter, the system would dynamically adjust this value based on observed task graph characteristics and historical performance.

Expected Benefits and Challenges

Expected improvements include 15-25% additional runtime reduction for applications with repetitive patterns like iterative optimization algorithms, where the ML model learns optimal CPU-GPU partitioning strategies specific to the problem domain. Energy efficiency should improve by 10-15% through more accurate worker sleep decisions, reducing wasteful steal attempts predicted to fail.

Implementation challenges would include the computational overhead of running ML inference on the critical scheduling path. To address this, I would use extremely lightweight models (under 10,000 parameters) implemented with quantized weights, ensuring prediction latency remains below 1 microsecond. Another challenge is the cold-start problem where the model has no training data initially. My solution would be to pre-train synthetic task graphs representing common patterns, then fine-tune during actual execution.

The key trade-off is complexity versus benefit. While pure work-stealing is simple and provably efficient, real-world applications often have exploitable patterns that static algorithms miss. By combining Taskflow's solid theoretical foundation with adaptive learning, the system could achieve best-of-both-worlds performance: guaranteed baseline efficiency with potential for significant pattern-specific optimization.

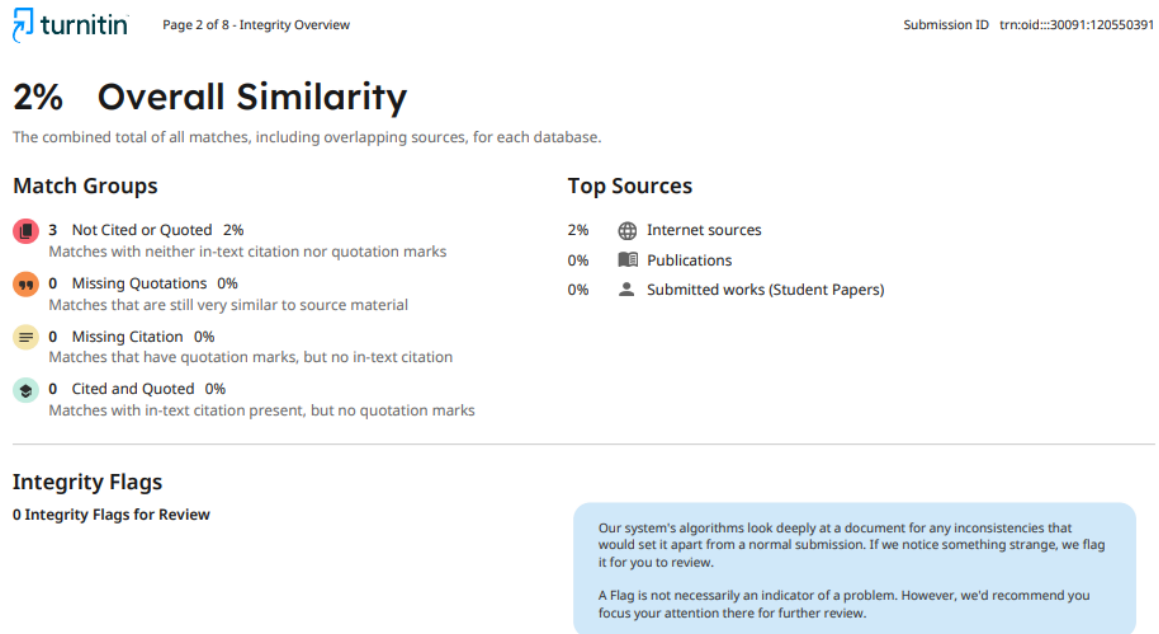


Figure 2 Turnitin Report